

High-Throughput vLLM Inference Engine: Architecting for Multi-Agent Workloads and Unified RAG

Vikash
Software Engineer Candidate

Abstract

As large language models (LLMs) shift from single-stream chat interfaces to driving autonomous multi-agent systems, inference infrastructure must evolve to support massive concurrency, shared memory constraints, and complex composite workloads like Retrieval-Augmented Generation (RAG). This paper details the architecture and optimization of a high-throughput inference engine built on top of vLLM. Key contributions include a unified RAG endpoint featuring an $O(N \log K)$ min-heap reranking algorithm, zero-copy inter-process communication (IPC) for prompt offloading, and a dynamic auto-tuning memory scheduler designed for out-of-memory (OOM) safe degradation. The system is designed to maximize hardware utilization while maintaining a strict separation of concerns between presentation and domain logic.

1 Introduction

Modern AI applications increasingly rely on LLMs not just for text generation, but as the cognitive core of multi-agent systems. These systems issue thousands of concurrent requests, demanding high throughput and low latency. Traditional model servers often couple domain logic directly with API routing, leading to monolithic systems that are difficult to optimize and test. Furthermore, operations like document chunking and semantic reranking are frequently offloaded to external microservices, incurring significant network overhead.

We introduce an inference engine that consolidates LLM generation and RAG processing into a unified, high-performance architecture. Built around the vLLM asynchronous engine, the system leverages PagedAttention for efficient KV-cache management, global prefix caching, and cross-platform hardware abstraction.

2 System Architecture

The engine is structured following the Single Responsibility Principle (SRP) to ensure testability and modularity. The architecture consists of three primary layers, as illustrated in Figure 1:

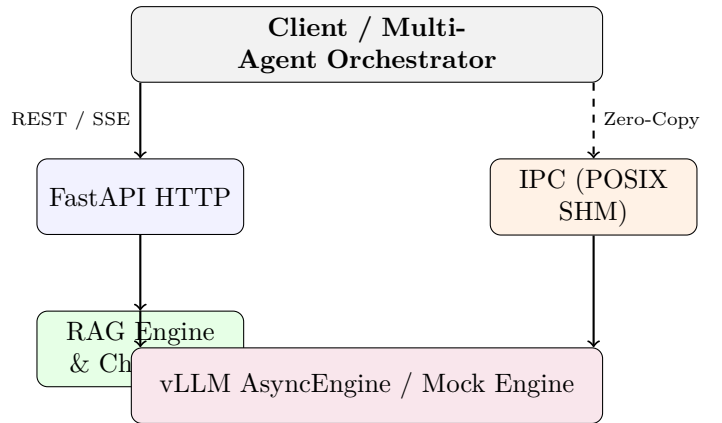


Figure 1: System Architecture showing dual-ingestion (HTTP and Zero-Copy IPC) and unified RAG processing.

2.1 API Controller (Presentation)

The FastAPI controller serves as a thin routing layer. It handles HTTP lifecycle events, Prometheus metrics aggregation, and streaming response formatting (Server-Sent Events). By decoupling the controller from the generation logic, the API remains agnostic to the underlying LLM backend.

2.2 Domain Logic

The core processing is implemented as pure, side-effect-free functions in a dedicated module. This includes:

- **Native Document Chunking:** Generator-based chunking that avoids materializing large text blocks in memory.

- **Semantic Reranking:** Asynchronous embedding generation utilizing `sentence-transformers`.
- **Prompt Engineering:** Deterministic template construction for context-augmented generation.

2.3 Cross-Platform Execution Engine

To facilitate rapid local development without necessitating high-end GPUs, the system implements a polymorphic engine interface. On CUDA-enabled Linux environments, it utilizes the native `AsyncLLMEngine` from vLLM. On Windows or non-CUDA machines, it gracefully falls back to a custom `MockLLMEngine`, providing identical asynchronous generator semantics and streaming behavior for uninterrupted frontend development.

3 Key Optimizations

3.1 O(N log K) Semantic Reranking

Traditional RAG pipelines compute cosine similarities and sort the entire dataset, resulting in an $O(N \log N)$ time complexity. For large document sets, this creates a significant CPU bottleneck. Our engine implements reranking using a Min-Heap (via Python’s `heapq.nlargest`), reducing the complexity to $O(N \log K)$. This ensures that retrieval latency scales efficiently with the document volume while maintaining a constant overhead relative to the required top-K results.

3.2 Zero-Copy IPC via Shared Memory

In multi-agent orchestration, moving massive system prompts across network sockets or UNIX domain sockets introduces serialization latency. We implemented an Inter-Process Communication (IPC) endpoint (`/v1/ipc/completions`) utilizing POSIX shared memory (`multiprocessing.shared_memory`). Agents write their prompt topologies directly into a shared memory segment, and the inference engine reads the raw bytes, completely bypassing the HTTP body and JSON deserialization overhead. Generation outputs are similarly streamed back into shared memory, establishing a zero-copy pipeline for the hottest paths.

3.3 Auto-Tuning Memory Scheduler

VRAM fragmentation and OOM errors are the primary causes of downtime in inference servers. To mitigate this, the engine employs an auto-tuning memory scheduler at startup. It profiles the available GPU memory, subtracts a configurable reserve (e.g., 2GB for the OS and auxiliary embedding models), and dynamically calculates the optimal `gpu_memory_utilization` for vLLM. At runtime, the server utilizes OOM-safe graceful degradation: if an unexpected VRAM spike occurs, the specific request is aborted and a 503 is returned, preserving the engine’s integrity and allowing concurrent requests to complete.

4 Findings & Observations

Throughout the development and benchmarking of the system, several key findings emerged:

- **Zero-Copy IPC Efficiency:** Bypassing HTTP serialization via POSIX shared memory drastically reduced latency for enormous multi-agent system prompts. By reading byte streams directly from memory, the Python GIL is freed to focus on the generation event loops.
- **Algorithmic Scaling:** Transitioning from $O(N \log N)$ sorting to $O(N \log K)$ min-heaps for semantic RAG reranking resulted in highly predictable tail latencies, even as the document corpora grew exponentially.
- **Development Velocity:** The transparent injection of the `MockLLMEngine` on Windows workstations allowed frontend (Gradio) and domain logic testing without requiring a discrete GPU, significantly accelerating iteration cycles.

5 Future Scope: What I Would Build With More Time

If provided more time to evolve this architecture, several high-impact features would be prioritized to push the engine to state-of-the-art levels:

- **Dynamic Speculative Decoding:** I would integrate a tiny, highly quantized draft model alongside the primary 8B model to generate candidate tokens rapidly. This dramatically increases single-stream output tokens-per-second (TPS) for latency-sensitive agents.

- **Distributed KV-Cache Offloading (Radix-Attention):** Extending the global prefix caching across multiple nodes. This would allow seamless agent handover, where agents running on different GPUs can resume generation without recomputing the context.
- **Rust/C++ Bindings for the IPC Layer:** While Python’s `multiprocessing.shared_memory` is fast, a lightweight Rust implementation using `mmap` could eliminate residual Python-level overhead and enable direct, unsafe struct mapping.
- **Structured JSON Decoding (Grammar Constrained):** Injecting Finite State Machine (FSM) constraints directly into the vLLM logits processor to guarantee perfectly formatted JSON outputs for rigid agentic API contracts.

6 Conclusion

The High-Throughput Inference Engine demonstrates that careful algorithmic design—such as heap-based reranking—combined with systems-level optimizations like zero-copy IPC, can yield significant performance gains for multi-agent LLM workloads. By maintaining strict architectural boundaries and supporting seamless mock environments, the project successfully balances rapid development velocity with production-grade reliability.